TrainWise

Full-Stack Fitness + Injury-Prevention Platform

React Native + Expo / ASP.NET Core 8 / Python ML / SQL Server

Architecture, ML, security and deployment, current.

Use this document as the canonical reference for any future Claude / Claude Code conversation on this project.

Author: Liron Vaknin

Generated: 2026-07-26

Document built by Claude Code (claude-opus-4-8)

Table of Contents

1. Project Overview
2. Technology Stack
3. Repository Layout
4. Architecture (4 tiers: App / API / ML / DB)
5. Backend Deployment Modes (Azure / Local LAN)
6. Authentication & Security
7. SQL Server Database (schema, migrations, seed)
8. AC-Ratio Training-Load Algorithm
9. Backend - C# ASP.NET Core 8
9.1 Controllers (29)
9.2 Business Logic (39) & Data Access (29)
9.3 Models (47)
10. Frontend - React Native Expo
10.1 Navigation tree (4 tabs)
10.2 Module layout & theme system
10.3 Screens (by group)
10.4 Health Connect integration
11. ML Service - Python / Flask (the smart element)
12. Feature Inventory (grouped)
13. End-to-End Data Flows
14. Build, Run & Deploy
15. Known Issues & Backlog
16. File Index & Change Log

This edition supersedes the 2026-04-16 draft. Major changes since then: JWT auth + security hardening, the intensityFactor removal, the dynamic cold-start load math, the Python ML service, and dozens of shipped features.

1. Project Overview

TrainWise is a full-stack fitness and injury-prevention platform for runners, gym-goers and their coaches. Its reason to exist is one sports-science idea: the Acute:Chronic Workload Ratio (ACWR), the relationship between the last 7 days of training (acute) and the 28-day baseline (chronic). Spiking that ratio is one of the best-known predictors of soft-tissue injury. TrainWise logs every workout (typed in by hand or synced automatically from Android Health Connect), computes the ACWR with a colour-coded Green / Yellow / Red warning, and layers a Python machine-learning service on top that forecasts where a trainee's load is heading and classifies overload risk.
Around that core sits a complete consumer app: coach/trainee linking and chat, a social layer (friends, gyms, live presence), gamification (badges, coins, streaks, shop, leaderboards), an AI assistant, nutrition, GPS run tracking, assigned training programs, a calendar, and more. It is a real three-tier system deployable fully on Azure or on a local LAN.

Primary user stories

- As an athlete, I log a workout and immediately see whether my load is in the safe zone or trending to overload (traffic light).
- As an athlete, my workouts import automatically from Health Connect so I do not have to enter them by hand.
- As an athlete, I report an injury and the system tightens my load thresholds for the recovery window.

- As a coach, I connect to a trainee (QR / coach-offer) and see their PMC, ACWR trend, monthly forecast and risk on one screen.
- As a coach or trainee, I run a 'what-if' simulation to see where my risk lands before I train the extra sessions.

At a glance

Author: Liron Vaknin
Tiers: Mobile app / REST API / ML service / SQL database
Scale: ~163 frontend files, ~145 C# files, ~10 ML modules, 32 SQL scripts
Features: ~109 shipped (see section 12); backlog in tasks/feature_backlog.md
Canonical docs: CLAUDE.md (engineering), PROJECT_SUMMARY.md (overview), this PDF

2. Technology Stack

Frontend

- React Native 0.81.5 + React 19.1.0, Expo SDK 54 (New Architecture required)
- JavaScript only (no TypeScript despite tsconfig.json)
- React Navigation v7 (native stack + bottom tabs)
- axios; @react-native-async-storage; react-native-svg (custom charts)
- react-native-reanimated / worklets; react-native-health-connect
- expo-maps / location / task-manager (GPS), camera, notifications, local-authentication, image-picker, audio, video, sharing
- @react-native-google-signin; react-native-qrcode-svg; webview (reCAPTCHA)

Backend

- ASP.NET Core 8.0 Web API, raw ADO.NET (Microsoft.Data.SqlClient)
- JWT bearer auth (Microsoft.AspNetCore.Authentication.JwtBearer)
- Built-in rate limiting; Swagger (Development only)
- FirebaseAdmin (FCM push); Google token + reCAPTCHA server verification

ML / Data Science

- Python 3.10+, Flask (port 8000), pandas, numpy
- scikit-learn: LinearRegression, PolynomialFeatures, RandomForest, KMeans
- matplotlib / seaborn (notebooks), joblib (model export)
- pyodbc (local, Windows auth) / pymssql + gunicorn (Azure)

Database & Cloud

- SQL Server: Azure SQL (cloud) or SQL Express Lirone\SQLEXPRESS (local)
- Azure App Service (API + ML), Azure SQL Serverless (auto-pause)
- External: Google Health Connect / Maps / Weather / Air-Quality / Places, Google Sign-In, OpenAI, Firebase Cloud Messaging, Open Food Facts

3. Repository Layout

The real project root is c:\Dev\TrainWise. Four cooperating sub-projects, no shared package manager.

```
TrainWise/                          <-- project root
  CLAUDE.md                         master engineering doc (592 lines)
  PROJECT_SUMMARY.md                full project overview
  generate_docs_pdf.py              THIS script (PyMuPDF)
  TrainWise_Project_Documentation.pdf   output of this script

  TrainWise/TrainWise/              <-- C# backend (TrainWise.sln)
    Controllers/   (29)   BL/  (39)   DAL/  (29)   Models/ (47)
    Program.cs   appsettings.json   wwwroot/images/
  TrainWise/TrainWise.Tests/        xUnit: pins the load-window math

  TrainWiseExpo/                    <-- React Native / Expo app
    App.js  index.js  app.json  app.config.js  eas.json  package.json
    android/                        native project (manual HC edits)
    src/  api/  services/  navigation/  screens/ (~60)
          components/ (~55)  utils/ (~55)  theme/  i18n/  config/

  ml/                               <-- Python ML microservice
    app.py db.py config.py features.py forecast.py risk.py auth.py
    models/*.pkl   notebook/*.ipynb (gradeable)
  ml_deploy_clean/                  clean copy for the Azure deploy

  sql/                              32 schema + migration + seed scripts
  tasks/                            session resumes, checklists, lessons.md
  Python Course ML/                 the 7-lesson ML course (context)
```

4. Architecture - 4 tiers

The mobile app talks to two services: the C# REST API for all app data, and the Python ML service for analytics/forecast. Both read the same SQL database. The C# backend keeps a strict three-layer shape.

```
  Mobile app (React Native)
    |  axios -> services/api.js  (all app data)
    |  axios -> services/mlApi.js (analytics / forecast / what-if)
    v                          v
  C# REST API               Python ML service (Flask :8000)
  Controller -> BL -> DAL       features / forecast / risk
    |  (ADO.NET)                 |  (pyodbc / pymssql)
    v                          v
  ===============  SQL Server  ===============
```

One formula, four implementations

The ACWR load math is the single source of truth in C# LoadCalculationBL (internal static helpers), and is mirrored byte-for-byte by ml/features.py (Python) and by utils/acwr.js + utils/loadSeries.js (on-device JS). The C# xUnit test project pins hand-computed vectors so the four never drift. Keeping them in lockstep is a defining challenge of the project.

Backend three-layer

Controllers are thin REST surfaces (Route api/[controller], [FromBody] on POST/PUT). BL holds business logic. DAL is manual ADO.NET over stored procedures (older features) or inline parameterised SQL (newer features). No EF Core, no migrations folder; the schema is managed by the scripts in sql/.

5. Backend Deployment Modes

Two interchangeable modes, flipped by a one-line switch plus an APK rebuild. BACKEND_MODE lives in src/config/backend.js (both axios clients import API_BASE_URL from it, so they can never drift). ML_MODE lives in src/services/mlApi.js. As of this writing both are 'local'.

Mode A - Azure (works anywhere)

- API on Azure App Service; Azure SQL (trainwiseadmin01.database.windows.net / TrainWiseDB).
- ML service live at trainwise-ml-...azurewebsites.net (/health -> db:true).
- DB connection injected via the App Service Connection-strings blade; DBservice.Connect() reads env vars so it wins over appsettings.json.
- Cost kept low by Azure SQL Serverless (auto-pause; first call after idle can take ~30s to wake).

Mode B - Local LAN

- API in VS 2022 (bind 0.0.0.0:5249), SQL Express, ML service (python app.py, :8000), phone over the same WiFi (or adb reverse over USB).
- Requires: firewall rules (TCP 5249 + 8000, Private profile), usesCleartextTraffic=true in the manifest, and LOCAL_PC_IP current.

> **WARNING: The C# backend code is identical in both modes**
>
> Nothing in Controllers/BL/DAL changes when switching. Only client config (backend.js / mlApi.js) and the DB connection source differ.

6. Authentication & Security

Since the 2026-07-02 security pass the API has a real identity layer (the old 'no auth, trusts a client-supplied userId' model is gone).

JWT bearer auth

- JwtService (HS256) mints a token on login / signup / google-login, carrying uid, isCoach/isTrainee, and a session id (sid).
- Program.cs validates a token when present. AUTH_ENFORCE (env flag) is the stage-2 switch: when true, every endpoint without [AllowAnonymous] requires a valid token. Off by default so older tokenless APKs keep working during rollout.
- BaseApiController exposes ownership gates (CallerMayAct, CallerOwnsOrCoaches, ...) that DENY a token belonging to a different user even while enforcement is off - closing IDOR/BOLA.
- Session revocation: OnTokenValidated checks sid against SessionBL.IsSessionActive, so 'log out this device' is real.

Other hardening

- PBKDF2 password hashing (PasswordHasher: SHA-256 / 100k / salted, verify-and-upgrade from legacy plaintext).
- Rate limiting (10/min/IP on auth endpoints, 300/min/IP global).
- Generic 500 body (never leak ex.Message / SQL / paths); the detail stays in the server log.
- Upload validation (UploadValidator: 6 MB cap + magic-byte sniff, server-derived extension; client filename ignored).
- Server-side Google ID-token verification (GoogleTokenVerifier) and signup reCAPTCHA (CaptchaVerifier, fail-open when unset).

> **WARNING: Secrets discipline**
>
> After a Google API key leaked via a push (now rotated), the project enforces a pre-commit secret scan and a safe-push checklist. Keys live in .env (gitignored) + app.config.js injection; appsettings.json carries the Azure SQL password locally but its committed version has only the clean local string and must never be committed with the password. EXPO_PUBLIC_* vars (the Maps + OpenAI keys) are baked into the APK in plaintext - do not distribute the APK publicly.

7. SQL Server Database

Raw ADO.NET against SQL Server. The base schema + stored procedures live in sql/TWDB.sql; 30+ dated migration scripts layer on the newer features. There is no EF migration runner, so every script must be run on BOTH the local and Azure databases.

7.1 Table families (roughly 40 tables)

- Core: Users, ActivityTypes, ActivityLogs, DailyLoad, LoadParameters, Recommendations.
- Coaching: Coaches, CoachTrainees, CoachRecommendations, WorkoutComments (coach feedback), MonthlyForecasts (ML snapshots).
- Injuries: InjuryTypes, InjuryCategories, InjuriesReports, InjuryPainLog.
- Messaging: Messages, MessageReactions, MessageTyping.
- Social: Friendships, Gyms, GymCoaches, CoachOffers (+ Users.LastSeen / Latitude / Longitude for presence + geo).
- Community / gamification: WorkoutBoard posts + likes + comments, WorkoutKudos, Challenges + participants, Events + RSVPs, cosmetics, records.
- Planning: PlannedWorkouts (calendar), TrainingPrograms / ProgramWorkouts / ProgramAssignments + program chat tables.
- Health & sessions: BodyMeasurements, NutritionLog, WorkoutTemplates, UserSessions, UserDevices (+ push token).

7.2 Stored procedures & inline SQL

Older features use ~49+ stored procedures (naming sp_VerbNoun); the newest features (programs, calendar, community, event chat) use inline parameterised SQL in the DAL. Both are parameterised - the project never string-concatenates user input into SQL.

7.3 Reference / seed data (required on any fresh DB)

sql/seed_reference_data.sql (idempotent) seeds the lookup tables the dropdowns and the load algorithm need: 20 ActivityTypes, 20 InjuryTypes + categories, 20 TrainingGoals, and the single LoadParameters tuning row. The social migration additionally seeds fake demo users + 10 real Netanya gyms (from Google Places). Without the seed, a fresh DB has empty dropdowns and a broken load calc.

8. AC-Ratio Training-Load Algorithm

Implemented in BL/LoadCalculationBL.cs (CalculateAndSave) and mirrored in Python + JS. This is the app's reason to exist.

Per-session load

```
sessionLoad = Duration (min)  x  Exertion (RPE 1-10)

NOTE: the old IntensityFactor multiplier was REMOVED across DB,
backend and frontend. Load is duration x exertion only.
```

Acute, chronic, ratio (mirrors LoadCalculationBL)

```
sessions bucketed per LOCAL calendar day (tzOffsetMinutes); pending
Health-Connect imports (IsConfirmed = 0) are EXCLUDED, NULL = confirmed

acute   = sum of session loads in the last 7 days
chronic = EffectiveChronic(28-day window)  (see below)
ratio   = acute / chronic     (NULL/Green when chronic = 0)
```

EffectiveChronic - dynamic cold-start floor + covered-days ramp

```
if < 7 active days in the 28-day window (cold start / layoff):
    chronic = max(sum28 / 4, experienceBootstrap)
    bootstrap weekly = Beginner 150 / Regular 280 / Advance 420
else (>= 7 active days):
    covered = days from the first loaded day through today (7..28)
    chronic = sum28 / min(4, covered / 7)     # ramp
```

The floor is DYNAMIC (based on the trailing window), not the one-shot IsBaselineEstablished flag - so a returning-from-layoff athlete no longer stores a false Red. The covered-days ramp stops a steady 2-week-old user reading a false 2.0; a full 28-day history is unchanged (/4).

Traffic-light classification (DetermineLoadLevel)

```
healthy:   ratio < 0.8 Green | 0.8..1.3 Yellow | > 1.3 Red
injured:   ratio < 0.8 Green | 0.8..<1.2 Yellow | >= 1.2 Red

Levels are graded from the UNROUNDED ratio (1.3049 is Red, not the
displayed 1.30). An active injury tightens the Red line with no gap.
```

9. Backend - C# ASP.NET Core 8

9.1 Controllers (29)

All follow the same shape: instantiate the matching BL, gate via BaseApiController, return Ok / BadRequest / generic 500.

- Auth & users: Auth, Users, UserDevices, Sessions
- Load & activity: ActivityLog, ActivityType, DailyLoad, LoadParameters, Recommendation
- Injuries: InjuryReport, InjuryTypes
- Coach: Coach, CoachTrainee, CoachRecommendations, Calendar, Programs
- Messaging & social: Messages, Social, Gyms, Community
- Workouts+: Nutrition, Records, WorkoutTemplates, WorkoutBoard, WorkoutComments
- Goals & prefs: TrainingGoals, UserGoals, UserActivityPreferences

9.2 Business Logic (39) & Data Access (29)

- Core load: LoadCalculationBL (the algorithm), LoadAnalyticsBL (rolling + EWMA trend), LoadParametersBL
- Auth/security services: JwtService, PasswordHasher, GoogleTokenVerifier, CaptchaVerifier, AuthRecoveryBL, SessionBL, InputValidator, UploadValidator, EmailSender
- Domain BL: User, ActivityLog, InjuryReport, Coach, CoachTrainee, Message, Social, Gym,

Nutrition, Records, Board, Community, Calendar, Program, WorkoutTemplate, WorkoutComment, Recommendation, CoachRecommendation, UserDevice
- Integrations: PushSender (FCM/FirebaseAdmin), PlacesService (Google Places)
- DAL: one per domain over DBservice.cs (Connect + sproc helper); newer DALs (Program, Calendar, Community, Board) use inline parameterised SQL.

9.3 Models (47)

POCOs shared between layers (User, ActivityLog, DailyLoad, LoadParameters, UserLoadContext, Coach*, Message*, Social*, Program*, Calendar*, Community*, Records*, Nutrition*, ...) plus a handful of Create*/Update* request objects. Unlike the 2026-04 draft, the models now carry the full column set (Email, IsCoach, IsTrainee, ExperienceLevel, baseline fields, ExertionLevel, Duration, CalculatedLoadForSession, IsConfirmed).

10. Frontend - React Native Expo

10.1 Navigation tree (4 tabs)

```
AppNavigator (auth vs app based on the session)
+-- AuthStack: Welcome -> Login | SignUp -> SignUpFinal
+-- AppTabs (Home / Load / Health / Connect)
      HomeTab   -> Home, Stats, Warnings, AddWorkout, Injury,
                   ActiveInjuries, WorkoutSummary/Route, Settings,
                   ConnectQR, Shop, AIChat/Plan, Chat, MyNetwork,
                   CoachTraineeDetail -> CoachTraineeAnalytics,
                   Profile, PersonalRecords, TrainingCalendar,
                   Programs, Timer, Achievements, LiveRun
      LoadTab   -> WarningsDashboard (load trend + analytics)
      HealthTab -> HealthConnect (GoogleFitScreen) + WorkoutRoute
      ConnectTab-> Connect, Requests, MyNetwork, Chat, Board,
                   Leaderboard, Feed, Challenges, Events, EventChat

Coach-only users see only Home + Connect (Load/Health hidden).
```

10.2 Module layout & theme system

- config/backend.js - the single BACKEND_MODE + API_BASE_URL source.
- api/ + services/ - AuthContext + JWT token store, the axios clients, Health Connect service + sync, messages/social contexts, weather + OpenAI + Google-auth helpers. services/mlApi.js = the ML client.
- theme/ - a MUTABLE Colors singleton swapped by applyTheme(); every screen must read it via the useThemedStyles(makeStyles) hook (a StyleSheet.create at module scope freezes colours and never theme-switches).
- i18n/ - EN / HE / FR translations foundation.
- utils/ (~55) - pure logic: acwr / loadSeries mirrors, badges, quests, calories, recovery, injuryRisk, achievements, etc.

10.3 Screens (by group)

- Auth: Welcome, Login, SignUp, SignUpFinal, ForgotPassword
- Load core: Home, HomeRouter, Stats, WarningsDashboard, WorkoutSummary; components LoadAnalyticsSection + AcwrTrendChart
- Workouts: AddWorkout, Timer, LiveRun, WorkoutRoute, ExerciseLibrary, PersonalRecords, NutritionScreen
- Injuries: InjuryReport, ActiveInjuries
- Coach: CoachDashboard, CoachTraineeDetail, CoachTraineeAnalytics (PMC/ACWR/forecast/what-if),

CoachPrograms, ProgramBuilder, CoachMarketplace
- Programs/calendar: TrainingCalendar, MyPrograms, ProgramDetail
- Social/community: Connect, ConnectQR, Requests, MyCoach (MyNetwork), WorkoutBoard, Leaderboard, Feed, Challenges, Events, EventChat, SharedWorkout
- Chat & AI: Chat, AIChat, AIPlan
- Gamification & misc: Shop, Achievements, Profile, Settings, Health Connect (GoogleFitScreen)

10.4 Health Connect integration

Read-only sync from Google Health Connect into backend ActivityLogs (HealthConnectService -> SyncService -> useSyncWorkouts). A persistent tombstone set stops deleted workouts re-importing. Getting the app to APPEAR in Health Connect on Android 14+/16 required a VIEW_PERMISSION_USAGE + HEALTH_PERMISSIONS activity-alias in the manifest (the months-long 'Android 16 wall'). Six read permissions; the app ships as a RELEASE APK (not Expo Go), and expo prebuild / EAS must NOT be used (they wipe the manual native edits).

11. ML Service - Python / Flask (the smart element)

A standalone Flask microservice (ml/app.py, port 8000) the app calls directly. It reads the same SQL database and mirrors the C# load formula exactly. It implements the spec in Python Course ML/TrainWise_Smart_Injury_Prevention_Updated.pdf and is the project's ML / Data-Science deliverable. It is live on Azure (trainwise-ml) but the app uses the local instance by default.

Endpoints

```
GET /health
GET /api/ml/trainee/<id>/pmc         Fitness / Fatigue / Form series
GET /api/ml/trainee/<id>/acwr        AC ratio + safe-zone thresholds
GET /api/ml/trainee/<id>/analytics   rolling + bias-corrected EWMA
GET /api/ml/trainee/<id>/forecast    monthly projection + risk (+snapshot)
GET /api/ml/trainee/<id>/forecast/history
GET /api/ml/trainee/<id>/whatif?addSessions=&intensity=easy|medium|hard
```

Task 1 - Regression (monthly forecast)

Splits the month into fixed weeks and fits the COMPLETED weeks: recent pace under 2 weeks, LinearRegression at 2, PolynomialFeatures(2) at 4+ only if it clearly fits better. Chronic is recomputed day-by-day in a forward simulation (so a rising acute is divided by a rising chronic and the ratio converges, instead of the old frozen-chronic bug that produced impossible ratios). Confidence = R-squared. Snapshots append to MonthlyForecasts, so a month refines weekly and past months stay reviewable. A global model (forecast_model.pkl, trained on documented synthetic data) runs alongside as a secondary comparison.

Task 2 - Classification (overload risk)

Labels each state Safe / Warning / High. Loads risk_model.pkl (a RandomForest chosen in the notebook over LogisticRegression) when present, else falls back to the threshold rule so the badge always renders. Features: AC ratio, acute, chronic, experience, age, active-injury count.

Charts, clustering, what-if

- PMC: Fitness (chronic) / Fatigue (acute) / Form (fitness - fatigue).
- ACWR chart: ratio over time, 0.8-1.3 safe band shaded, 1.5 danger line.
- KMeans clustering segments trainees by load profile.
- What-if: injects N easy/medium/hard sessions (150/300/450) onto today and recomputes with the SAME rolling math; verified +3 hard: 1.06 Warning -> 1.93 High.

Notebooks (gradeable, course-style)

ml/notebook/*.ipynb follow the course flow: clean, EDA (seaborn heatmaps), regression (MAE/MSE/RMSE/R2 + residuals), classification (Accuracy/Precision/Recall/F1 + confusion matrix +

multiclass ROC/AUC), KMeans, and joblib export. Real data drives the charts; synthetic data trains the global models, and the split is documented for the grader.

> **Sports-science basis**
>
> ACWR (Gabbett 2016), bias-corrected EWMA (Williams 2017 + the Adam zero-init correction, Kingma & Ba 2015), the PMC fitness-fatigue model, and Foster's monotony & strain (1998). Real, cited methods.

12. Feature Inventory (grouped)

~109 shipped features. Highlights by area:

Load & injury core

ACWR dashboard (Home + Load tab), Warnings dashboard, load-trend analytics (rolling + EWMA), injury reporting + active tracking + mark-recovered, body-map picker, pain logging, rehab suggestions, injury-risk gauge.

Workouts & wearables

Manual add-workout, Health Connect sync, workout templates, interval timer, live GPS run (background tracking), HR zones, CSV/PDF export, notes/photos, exercise library, personal bests, deep-link share.

Coach / trainee

QR + coach-offer linking, per-trainee dashboard, ML analytics (PMC/ACWR/forecast/what-if), assigned programs (fan out to calendar), coach comments, video form-check, progress reports, marketplace + reviews.

Social & community

Friends, gyms map (real Netanya gyms), live presence, workout board + comments + kudos, activity feed, friend challenges, group events, leaderboards + seasonal divisions.

Chat, gamification, AI, health

User chat (text/image/voice) + reactions + typing + group chat + FCM push; badges + coins + shop + streaks + quests + confetti; weather smart card, AI week-in-review / plan / ask-my-data / injury-photo advice; nutrition + barcode + calorie ring, weight/body composition, sleep/HRV readiness.

Platform / UX

Dark + light themes + accent picker, i18n (EN/HE/FR), biometric login, forgot/reset password + email verification, multi-device sessions, notification preferences, What's-New changelog, accessibility pass.

13. End-to-End Data Flows

Add workout

```
AddWorkoutScreen (or LiveRun / Health Connect confirm)
  -> services/api.createActivityLog(payload)  # duration x exertion
       POST /api/activitylog -> ActivityLogBL -> ActivityLogDAL
  -> services/api.calculateDailyLoad(userId, date, tzOffsetMinutes) x2
       POST /api/dailyload/user/{id}/calculate
         LoadCalculationBL.CalculateAndSave
           bucket-by-local-day, acute 7d, EffectiveChronic 28d,
           ratio, level, stress -> sp_SaveDailyLoad
  -> navigate WorkoutSummary { summary }
```

Coach analytics / forecast (ML)

```
CoachTraineeAnalyticsScreen (also the trainee's 'My analytics')
  -> services/mlApi.getTrainee{Pmc|Acwr|Analytics|Forecast|WhatIf}
       GET http://<host>:8000/api/ml/trainee/{id}/...
         features.py (rolling + EWMA) / forecast.py / risk.py
         reads ActivityLogs directly (never stale DailyLoad rows)
  -> SVG charts (PMC, ACWR safe-zone), forecast card, what-if planner
  If the ML service is unreachable, the screen degrades to the C#
  fallback (LoadAnalyticsBL) and then the on-device JS mirror.
```

14. Build, Run & Deploy

Database

- Run sql/TWDB.sql, then the dated migrations in order, then seed_reference_data.sql (see CLAUDE.md for the exact run-order). Run on BOTH local and Azure DBs.

Backend

- Open TrainWise.sln in VS 2022, run (Swagger at https://localhost:5249/swagger in Development). For Azure: right-click Publish.
- Local LAN: bind 0.0.0.0:5249, firewall TCP 5249 (Private).

ML service

```
cd ml
python -m venv venv & venv\Scripts\activate
pip install -r requirements.txt
python app.py            # http://0.0.0.0:8000  (firewall TCP 8000)
# Azure: deploy the ml_deploy_clean folder; 4 App Settings for SQL auth
```

Frontend APK

```
cd TrainWiseExpo
npm install
npx expo run:android --variant release     # or:
cd android & gradlew assembleRelease        # delete app/.cxx + app/build
                                            # first to force a fresh build
# output: android/app/build/outputs/apk/release/app-release.apk
```

> **WARNING: Do NOT run expo prebuild / EAS Build**
>
> Both regenerate android/ from app.json + plugins and WIPE the manual Health-Connect manifest edits (the activity-alias for Android 14+), which silently breaks HC. Use gradlew / expo run:android instead. Verify a fresh APK by its timestamp, not the 'BUILD SUCCESSFUL' line.

15. Known Issues & Backlog

WARNING: ML service is local-only by default

mlApi.js ML_MODE defaults to 'local', so the coach analytics, trainee Load Trend and the what-if planner need the local Python service reachable on the same WiFi. The code is Azure-ready (ml/db.py dual-mode, the trainwise-ml resource is live); flip ML_MODE to 'azure' + rebuild to use the cloud.

WARNING: mlApi.js hardcodes its own ML IP

It carries its own ML_BASE_URL instead of importing LOCAL_ML_URL from config/backend.js, so the PC-IP change is a two-file edit (backend.js AND mlApi.js) rather than one.

WARNING: AI key shipped in the APK

The OpenAI + Google Maps keys use the EXPO_PUBLIC_ prefix, so they are baked into the APK in plaintext. Fine for the demo; do not distribute the APK publicly. Production fix: proxy the calls through the backend.

WARNING: wwwroot/images on Azure

Profile-pic + chat-image uploads write to wwwroot/images, which Azure App Service may wipe on restart. Migrate to Azure Blob Storage or a persisted disk for production persistence.

Resolved since the 2026-04 draft: the login/AuthController now exists (JWT), passwords are PBKDF2-hashed (not plaintext), the controller-attribute and model-column gaps are closed, intensityFactor is removed, and the two axios clients now share one API_BASE_URL. The forward backlog (IDs 110-185) lives in tasks/feature_backlog.md.

16. File Index & Change Log

Key documents

CLAUDE.md master engineering doc (architecture, HC rules, secrets, deploy)
PROJECT_SUMMARY.md full project overview
tasks/lessons.md the self-learning log (~90 entries)
tasks/feature_backlog.md forward backlog (IDs 110-185)
sql/seed_reference_data.sql required lookup + demo seed
ml/notebook/*.ipynb gradeable ML notebooks

What changed in this edition (vs 2026-04-16)

- Added: JWT auth + security hardening (section 6).
- Added: the Python ML service (section 11) and the 4-tier architecture.
- Added: deployment modes, the feature inventory, and the corrected cold-start load math.
- Corrected: intensityFactor removed; counts (29 controllers, ~60 screens, ~40 tables); release APK (not Expo Go); PBKDF2 passwords.
- Removed: obsolete 'no auth' / 'broken controller' / 'plaintext password' warnings (all resolved).

End of document

Regenerate at any time: py generate_docs_pdf.py